

Exploratory Data Analysis (EDA) is all about asking the right questions, finding answers through data, and presenting them with clear visualizations. After cleaning the dataset containing information about passengers who boarded the ship in the last practice, let's build today's EDA step by step.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
In [2]: df = pd.read_csv('D:\Python For data Analysis - IBM\To upload on linkedin\Day1-22nd Sept-Data wrangling\\Passengers_clean.csv')
```

```
In [4]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Unnamed: 0      891 non-null   int64
1   PassengerId     891 non-null   int64
2   Survived        891 non-null   int64
3   Pclass          891 non-null   int64
4   Passenger_Name  891 non-null   object
5   Sex             891 non-null   object
6   Age            891 non-null   float64
7   SibSp           891 non-null   int64
8   Parch          891 non-null   int64
9   Fare           891 non-null   float64
10  Embarked        889 non-null   object
11  AgeGroup        817 non-null   object
dtypes: float64(2), int64(6), object(4)
memory usage: 83.7+ KB
```

The newly created 'AgeGroup' column still contains some missing values after processing.

Yesterday, we assigned the label 'Adult' to ages between 30 and 50. However, our data includes several records with ages greater than 50, which were not captured by this binning logic and thus remain unlabeled.

We need to resolve the remaining missing values in the 'AgeGroup' column before performing exploratory data analysis (EDA), to ensure accurate and meaningful insights

```
In [5]: df.columns
```

```
Out[5]: Index(['Unnamed: 0', 'PassengerId', 'Survived', 'Pclass', 'Passenger_Name',
              'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked', 'AgeGroup'],
              dtype='object')
```

```
In [6]: # We will check the total missing values in 'AgeGroup'
df['AgeGroup'].isnull().value_counts()
```

Out[6]: AgeGroup  
False 817  
True 74  
Name: count, dtype: int64

```
In [7]: # We check and found that all missing values are above 30.
missing_data = df[df['AgeGroup'].isnull()][ 'Age' ]
count_of_missing_values_above_30 = (missing_data < 30) .count()
print(f"There are total {count_of_missing_values_above_30} values, which are above 30 Age.")
```

There are total 74 values, which are above 30 Age.

```
In [8]: # Now We will replace, all missing values in 'AgeGroup' column by 'Adult' label.
df['AgeGroup'].replace(np.nan, 'Adult', inplace = True)
```

C:\Users\Dell\AppData\Local\Temp\ipykernel\_4016\324333421.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['AgeGroup'].replace(np.nan, 'Adult', inplace = True)
```

```
In [9]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Unnamed: 0      891 non-null   int64
1   PassengerId     891 non-null   int64
2   Survived        891 non-null   int64
3   Pclass          891 non-null   int64
4   Passenger_Name  891 non-null   object
5   Sex             891 non-null   object
6   Age             891 non-null   float64
7   SibSp           891 non-null   int64
8   Parch           891 non-null   int64
9   Fare            891 non-null   float64
10  Embarked        889 non-null   object
11  AgeGroup        891 non-null   object
dtypes: float64(2), int64(6), object(4)
memory usage: 83.7+ KB
```

```
In [11]: # Saving this into csv file for our future analysis.
df.to_csv('D:\Python For data Analysis - IBM\To upload on linkedin\Day2-24th Sept-EDA\clean_data.csv')
```

```
In [12]: # Great, now we have clean dataset. Lets perform EDA.
```

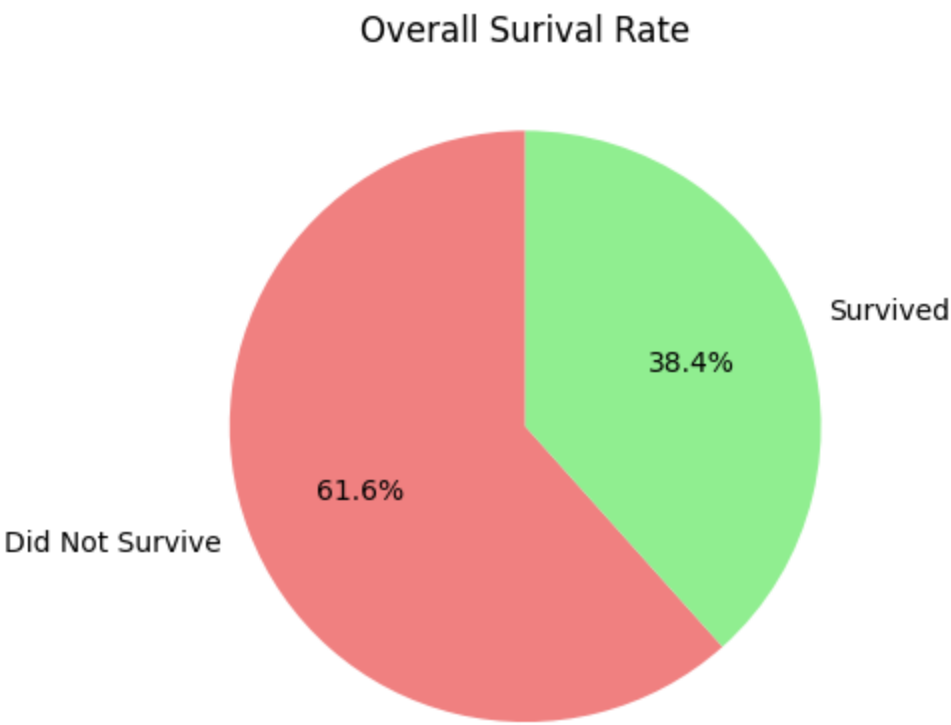
# 1. What is the overall survival rate?

```
In [13]: # We have a column named, 'Survived'. Using this column we can calculate the overall survival rate.
```

```
In [68]: # Value count of survived.
survival_counts = df['Survived'].value_counts(normalize = True) * 100 #Used normalize to show values in percentage.
print(f" About {survival_counts[0]:.1f}% did not servive.")
print(f" About {survival_counts[1]:.1f}% servive.")
```

About 61.6% did not servive.  
About 38.4% servive.

```
In [15]: # Visualization - Pie Chart.
plt.pie(df['Survived'].value_counts(),
        labels = ['Did Not Survive', 'Survived'],
        autopct = '%1.1f%%',
        colors = ['lightcoral', 'lightgreen'],
        startangle=90)
plt.title('Overall Survival Rate')
plt.show()
```



# 2. Does gender affect survival chances?

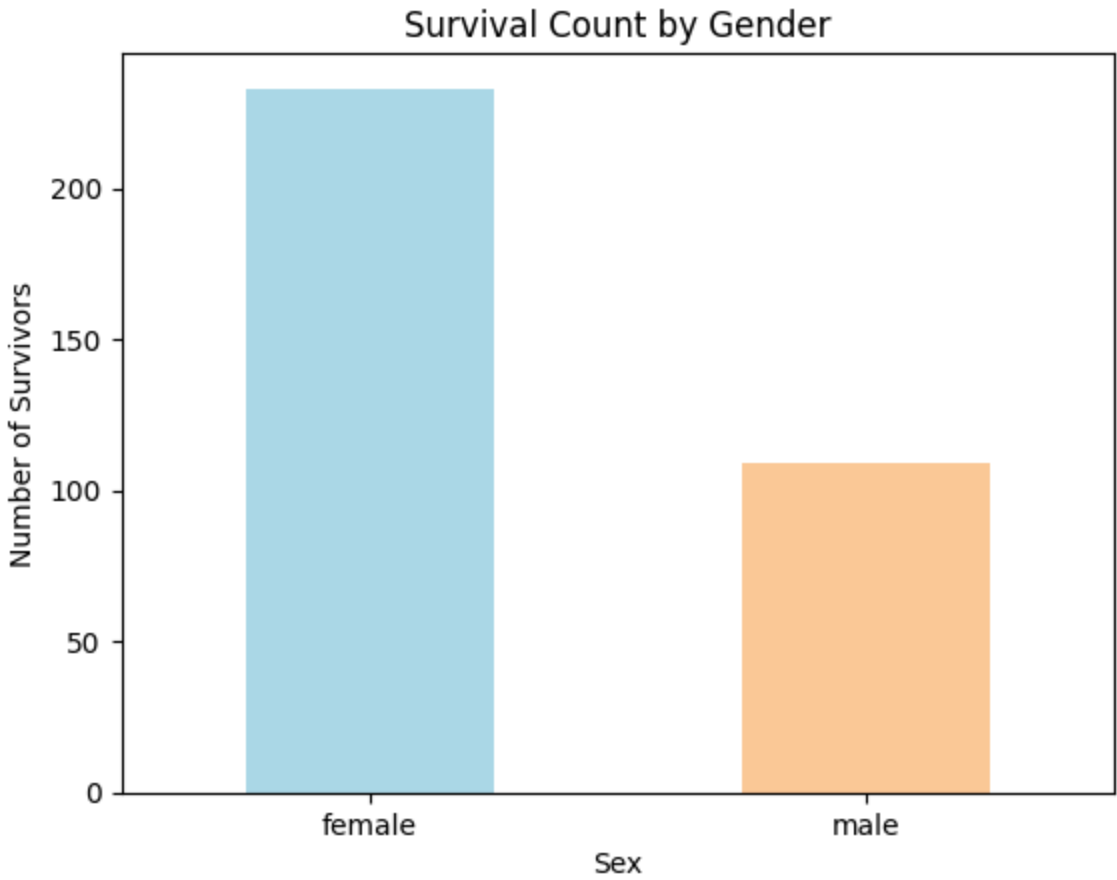
```
In [16]: # Group by gender and calculate survival rate
```

```
In [17]: gender_survival = df.groupby('Sex')['Survived'].mean()*100
print(f"Female Servival Rate {gender_survival[0]:.1f}%")
print(f"Male Servival Rate {gender_survival[1]:.1f}%")
```

Female Survival Rate 74.2%  
Male Survival Rate 18.9%

```
C:\Users\Dell\AppData\Local\Temp\ipykernel_4016\1143142463.py:2: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`  
    print(f"Female Survival Rate {gender_survival[0]:.1f}%")  
C:\Users\Dell\AppData\Local\Temp\ipykernel_4016\1143142463.py:3: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`  
    print(f"Male Survival Rate {gender_survival[1]:.1f}%")
```

```
In [18]: # Visualization - bar Chart.  
df.groupby('Sex')['Survived'].sum().plot(kind = 'bar',color = ['lightblue','#FAC898'])  
plt.title('Survival Count by Gender')  
plt.ylabel('Number of Survivors')  
plt.xticks(rotation=0)  
plt.show()
```



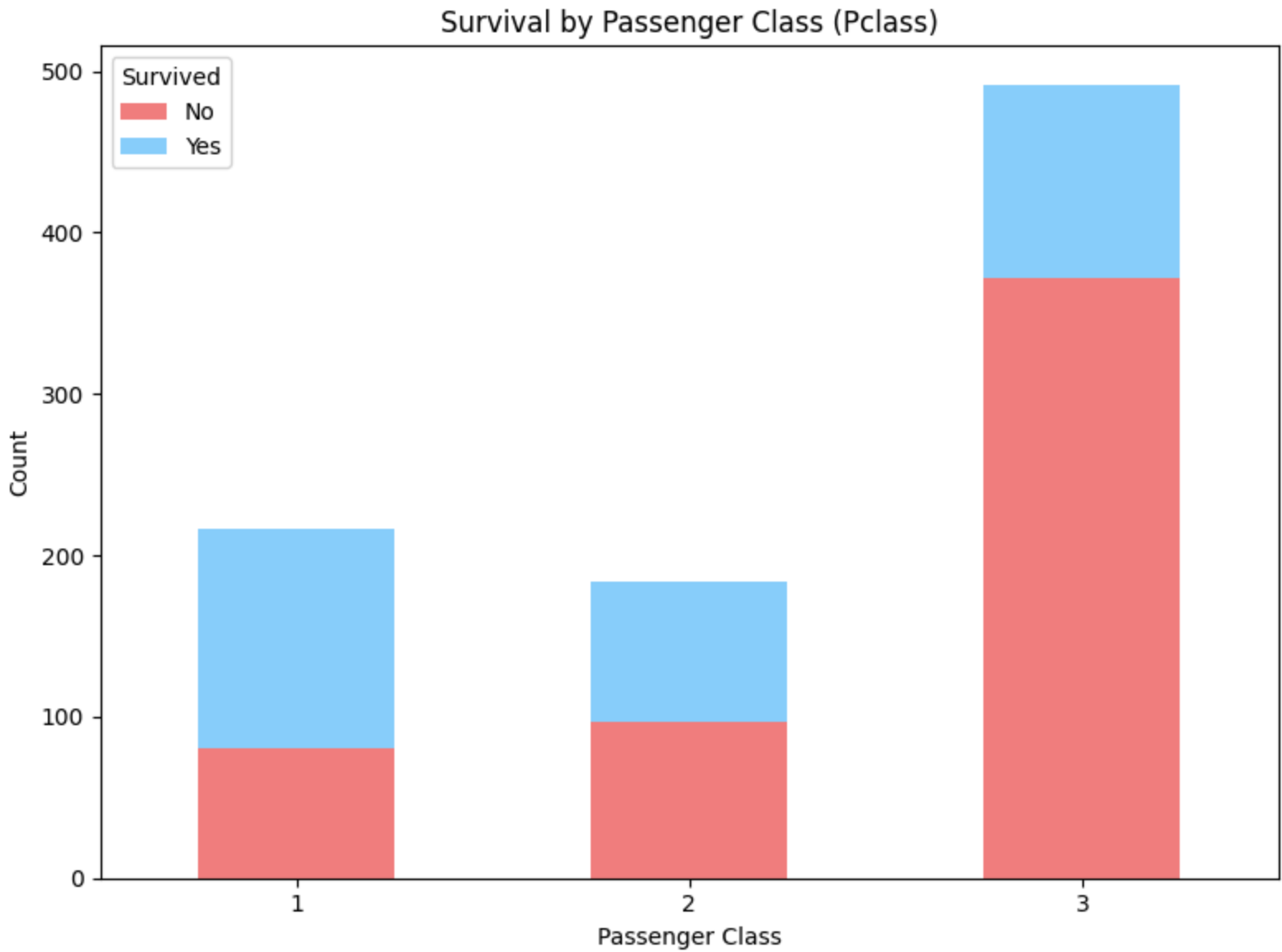
💡 Insight: Females had a much higher chance of survival, reflecting the "women and children first" evacuation policy.

3. What is the impact of passenger class (Pclass) on survival?

```
In [19]: pclass_survival = df.groupby(['Pclass', 'Survived']).size().unstack(fill_value=0)  
print(pclass_survival)
```

|          |     |     |
|----------|-----|-----|
| Survived | 0   | 1   |
| Pclass   |     |     |
| 1        | 80  | 136 |
| 2        | 97  | 87  |
| 3        | 372 | 119 |

```
In [20]: pclass_survival.plot(kind = 'bar',stacked = True,color = ['lightcoral','lightskyblue'],figsize=(8,6))
plt.title('Survival by Passenger Class (Pclass)')
plt.xlabel('Passenger Class')
plt.ylabel('Count')
plt.xticks(rotation=0)
plt.legend(title='Survived', labels=['No', 'Yes'])
plt.tight_layout()
plt.show()
```



💡 Insight: Higher-class passengers had much better survival chances. Wealth/status influenced access to lifeboats.

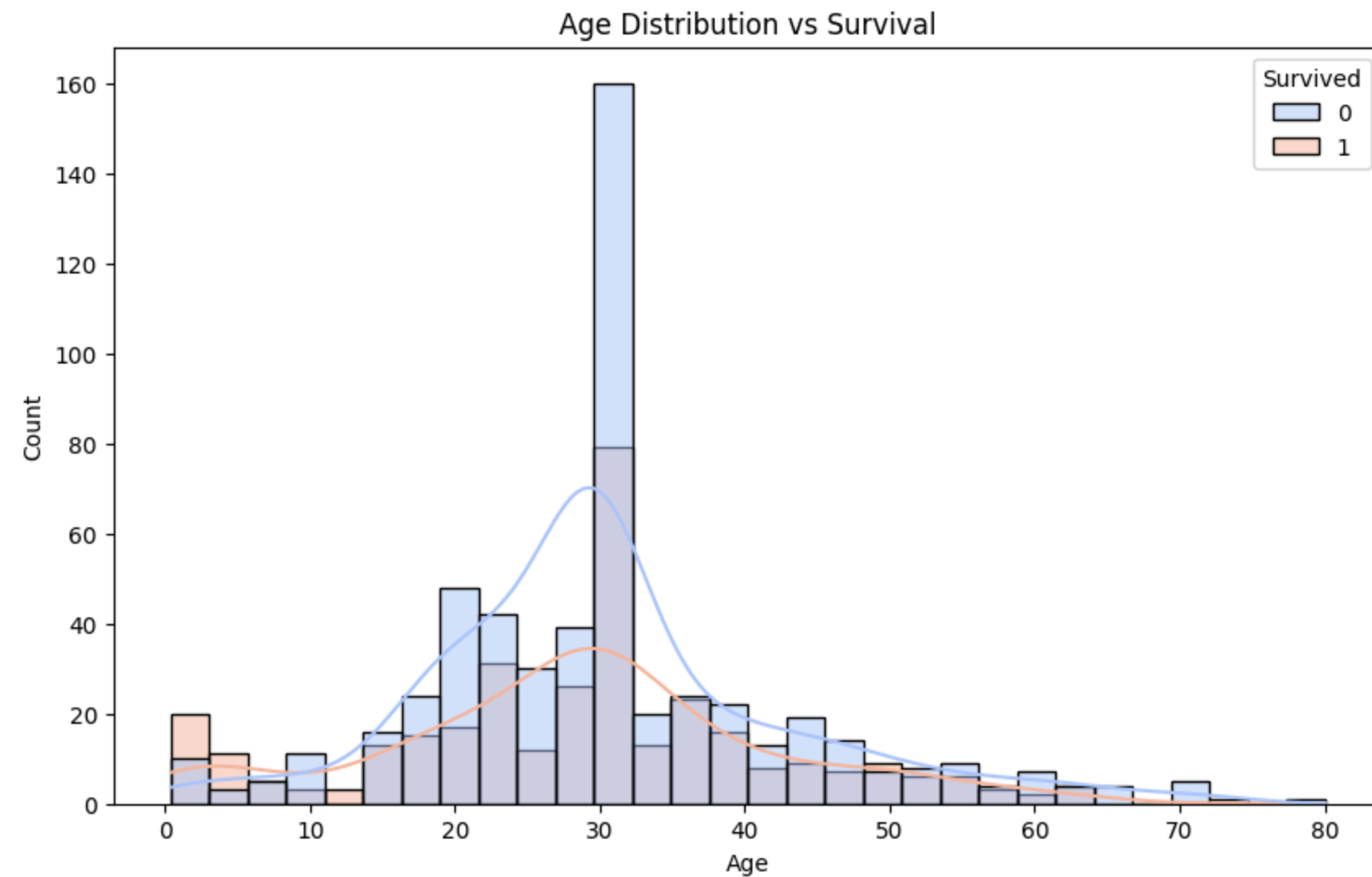
4. How does age affect survival?

```
In [49]: avg_age_survived = round(df[df['Survived'] == 1]['Age'].mean())
print(f"avg_age_survived = {avg_age_survived}")
avg_age_not_survived = round(df[df['Survived'] == 0]['Age'].mean())

print(f"avg_age_not_survived = {avg_age_not_survived}")
```

```
avg_age_survived = 29
avg_age_not_survived = 30
```

```
In [52]: plt.figure(figsize=(10,6))
sns.histplot(data = df,x='Age',hue="Survived",bins = 30,kde = True,palette = "coolwarm")
plt.title("Age Distribution vs Survival")
plt.show()
```



💡 **Insight:** Age had a weak effect overall, but children (<15) had noticeably better survival chances.

## 5. What is the survival distribution by family size?

```
In [55]: df['Familysize'] = df['SibSp'] + df['Parch'] + 1
```



```
In [62]: family_survival = df.groupby("Familysize")["Survived"].mean()*100
print(family_survival)
```

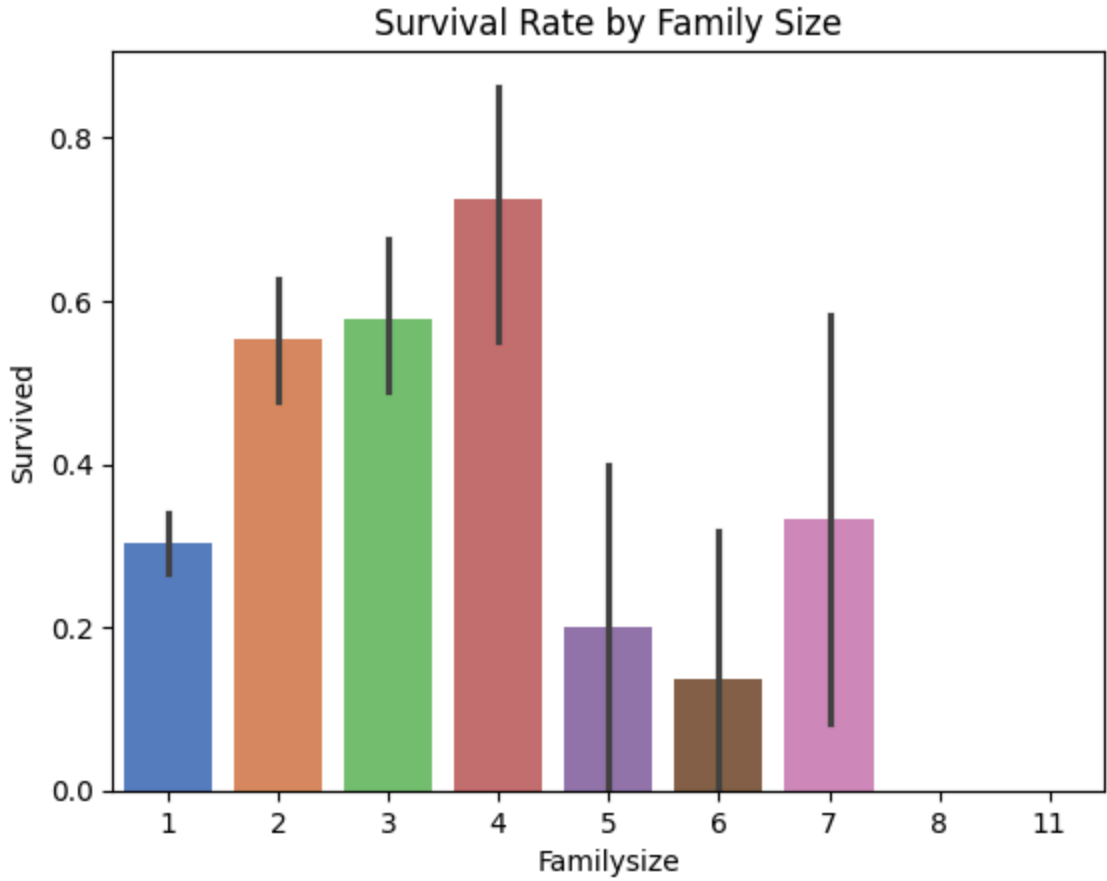
```
Familysize
1    30.353818
2    55.279503
3    57.843137
4    72.413793
5    20.000000
6    13.636364
7    33.333333
8     0.000000
11    0.000000
Name: Survived, dtype: float64
```

```
In [66]: sns.barplot(x="Familysize", y= "Survived",data = df,palette = "muted")
plt.title("Survival Rate by Family Size")
plt.show()
```

C:\Users\Dell\AppData\Local\Temp\ipykernel\_4016\135530834.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x="Familysize", y= "Survived",data = df,palette = "muted")
```



💡 **Insight:** Having a small family increased chances (support system), while being alone or in very large families reduced chances.

## 6. Did the embarkation port affect survival?

```
In [83]: df_embarked = round(df.groupby('Embarked')['Survived'].mean()*100)
print(f"Survival chances when embarked from Cherbourg (C) {df_embarked[0]:.1f}%")
print(f"Survival chances when embarked from Queenstown (Q) {df_embarked[1]:.1f}%")
print(f"Survival chances when embarked from Southampton (S) {df_embarked[2]:.1f}%")
```

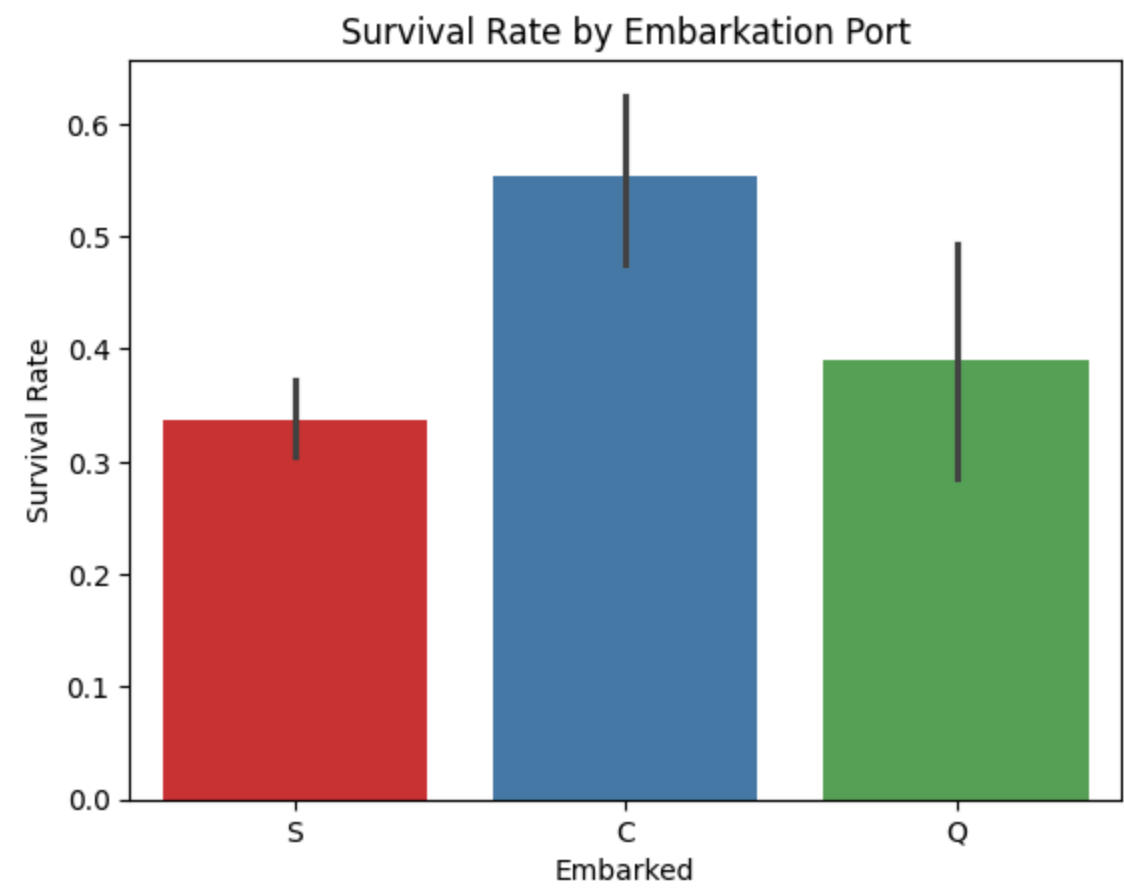
Survival chances when embarked from Cherbourg (C) 55.0%  
Survival chances when embarked from Queenstown (Q) 39.0%  
Survival chances when embarked from Southampton (S) 34.0%

C:\Users\Dell\AppData\Local\Temp\ipykernel\_4016\2819486339.py:2: FutureWarning: Series.\_\_getitem\_\_ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`  
print(f"Survival chances when embarked from Cherbourg (C) {df\_embarked[0]:.1f}%")  
C:\Users\Dell\AppData\Local\Temp\ipykernel\_4016\2819486339.py:3: FutureWarning: Series.\_\_getitem\_\_ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`  
print(f"Survival chances when embarked from Queenstown (Q) {df\_embarked[1]:.1f}%")  
C:\Users\Dell\AppData\Local\Temp\ipykernel\_4016\2819486339.py:4: FutureWarning: Series.\_\_getitem\_\_ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`  
print(f"Survival chances when embarked from Southampton (S) {df\_embarked[2]:.1f}%")

```
In [84]: sns.barplot(x='Embarked',y='Survived',data=df, palette="Set1")
plt.title("Survival Rate by Embarkation Port")
plt.ylabel("Survival Rate")
plt.show()
```

C:\Users\Dell\AppData\Local\Temp\ipykernel\_4016\4145039963.py:1: FutureWarning:  
  
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.  
  
sns.barplot(x='Embarked',y='Survived',data=df, palette="Set1")





💡 Insight: Passengers from Cherbourg had higher survival, likely because many were wealthier first-class travelers.

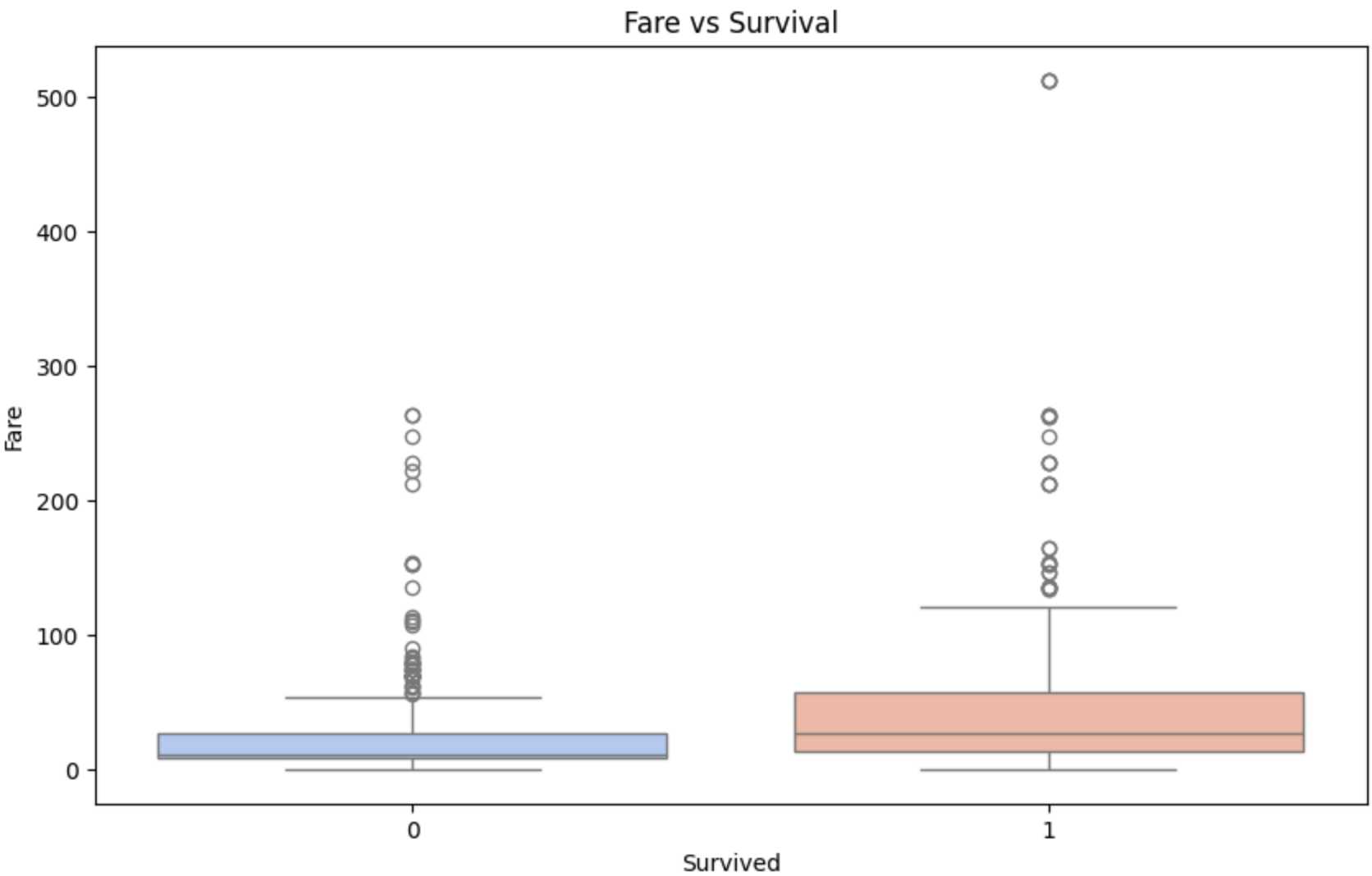
7. Is there a relationship between fare and survival?

```
In [94]: avg_fare_survived = df[df['Survived']==1]['Fare'].mean()
avg_fare_not_survived = df[df['Survived']==0]['Fare'].mean()
print("Avg Fare (Survived):",round(avg_fare_survived))
print("Avg Fare (Not Survived):",round(avg_fare_not_survived))
```

Avg Fare (Survived): 48  
Avg Fare (Not Survived): 22

```
In [99]: plt.figure(figsize=(10,6))
sns.boxplot(x="Survived", y="Fare", data=df, palette="coolwarm")
plt.title("Fare vs Survival")
plt.show()
```

C:\Users\Dell\AppData\Local\Temp\ipykernel\_4016\2166980179.py:2: FutureWarning:  
  
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.  
  
sns.boxplot(x="Survived", y="Fare", data=df, palette="coolwarm")



💡 Insight: Higher fare = higher survival probability, reinforcing the effect of passenger class and wealth.

8. Heatmap of correlations – which features are related to survival?

```
In [102... corr = df.corr(numeric_only=True) # only keeps numeric columns
print(corr["Survived"].sort_values(ascending=False))
```

```
Survived    1.000000
Fare        0.257307
Parch       0.081629
Familysize  0.016639
Unnamed: 0  -0.005007
PassengerId -0.005007
SibSp       -0.035322
Age         -0.069809
Pclass      -0.338481
Name: Survived, dtype: float64
```

Answer: Top correlations with survival:

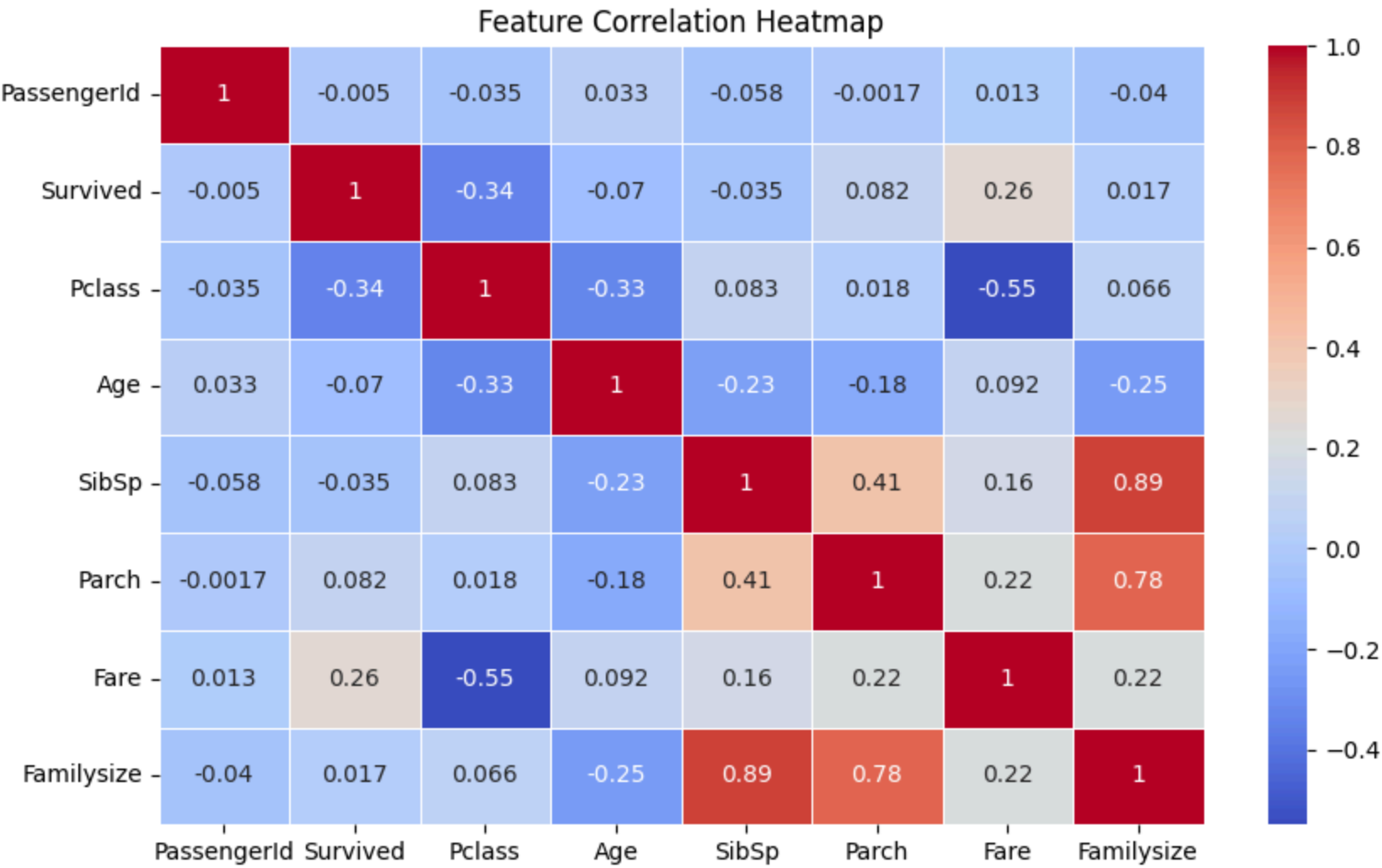
Fare (positive)

– Pclass (negative, higher class = better survival)

Sex (Female strongly linked with survival)

Weak correlation: Age, SibSp, Parch

```
In [107... plt.figure(figsize=(10,6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm", linewidths=0.5)
plt.title("Feature Correlation Heatmap")
plt.show()
```



💡 **Insight:** The most important predictors of survival were gender, passenger class, and fare. Other factors like age and family had weaker influence.

```
In [106... 
```

```
In [ ]: 
```